

3D 드라이브 게임: 스트리밍 월드 재설계와 게임성 강화안

도로·주변환경 선로딩 문제 진단 · 프레임 드랍/로딩 지연 근본 해결 설계 · 신규 게임 기능 | 2026-10-05 · 작성: Claude Code

1. 요약

핵심 진단. 현재 드라이브는 '단지 1개 = 장면 1개' 구조 위에서 동작한다. 그려진 영역(반경 약 600~680m) 끝에 가까워지면 다음 단지의 장면 전체(외벽 페인트·지형·물·식재·교통·가로등·노면표시)를 주행 중에 처음부터 다시 만들고, 완성되면 차량을 새 장면으로 옮기는 재중심화(hop) 방식이다. 선로딩은 데이터 요청까지만 앞당기고, 지오메트리 생성·GPU 업로드·셰이더 준비는 여전히 진입 직전에 몰린다. 그 결과 고속 주행 중 3~17초짜리 장면 생성이 렌더 프레임과 경쟁하면서 프레임 드랍, 월드 끝 노출, 충돌 판정 공백, 소리 끊김이 반복된다.

해결 방향. 장면 단위 재구축을 버리고 고정 격자 타일 스트리밍 월드로 바꾼다. 250m 타일, 부동 원점(floating origin), 워커 풀에서 타일 생성, 프레임당 업로드 예산, 주행 전용 사전 컴파일 파이프라인 세트, 도로 그래프 기반 ETA 예측 선로딩, 끊김 없는 연속 교통·충돌·오디오로 구성한다. 단지 상세(사진 기반 외벽)는 정차·저속일 때 덧입히는 업그레이드 층으로 분리해 주행이 그것을 기다리지 않게 한다.

목표 지표 (데스크톱, 프로덕션 빌드)	현재(코드·기록 기준 추정)	목표
180km/h 연속 주행 프레임 p99	재중심화 구간마다 장면 생성으로 긴 프레임 다수	≤ 20ms (CPU 4배 감속 ≤ 33ms)
월드 끝 노출 / 다음 영역 대기	고속 시 생성 3~17s > 도착 여유 약 5s	0회 (예측 여유 ≥ 2배)
다음 영역 진입 시 정지	차량 이관·교통 재생성·충돌 격자 재계산	0회 (연속 시스템)
주행 시작까지	solidGrid 계산 중 건물 충돌 없음	≤ 1s, 충돌 즉시 유효
30분 주행 메모리	두 장면 동시 상주 구간 반복	평탄(타일 상한 고정)

2. 현재 구조

2.1 주행 흐름 (ComplexHologram.tsx + driveSim/driveGame/driveFeel/driveFx/heroVehicles)

- 진입:** 쿠팡트럭·사이버트럭 버튼(?devgame=1일 때만 표시) → 교통이 운전하다가 키/터치 페달 입력 시 DriveSim이 넘겨받는다. 추적 시점/운전석 시점(V)을 지원한다.
- 시뮬레이션:** 자전거 모델, 자동 변속, 공기·구름 저항, 부스트, 120Hz 하위 스텝. 충돌은 차량 OBB(SAT), 가로수·가로등(PoleGrid 8m 버킷), 건물(SolidGrid 0.5m 래스터)로 판정한다.
- 게임:** 무작위 주변 단지로 배송, 녹색 리본 경로, 빗기둥, 턴바이턴 음성, 미니맵, 연료·내구도·픽업, 거리 비례 골드, Turso 점수판.
- 연속 주행(작업 트리 WIP):** 700ms 간격 타이머가 앞쪽 지점과 주변 단지 후보를 고르고, lookAhead/vworldPointArea로 데이터를 미리 받는다(areaJobs). 앞쪽 8초 지점이 중심에서 420m를 넘으면 setHop으로 재중심화하고, 옛 장면은 carry로 계속 그린다. 새 교통이 뜨면 carryRef가 차량·배송·픽업을 이관한 뒤 옛 장면을 해제한다.

그림 1. 현재 재중심화(hop) 흐름 (빨간 글씨 = 프레임 드랍/공백 발생 지점)



3. 문제 진단

#	문제	코드 근거	플레이어가 겪는 현상
D1	영역 이동 = 장면 전체 재구축	setHop → complexId effect 전체 재실행. 외벽 페인트·지형·물·식재·교통·가로등·노면표시를 다시 만든다. 코드 주석상 '고속에서 단지 장면 약 17s, 일반 영역 약 3s'	주행 중 수 초간 프레임 드랍, 순간 정지
D2	선로딩이 데이터 단계에서 멈춤	areaJobs는 응답(JSON)과 prefetchPaint만 미리 한다. 지오메트리·업로드·파이프라인은 due 이후에 시작한다	'불러옵니다' 토스트 후 한참 뒤 생성
D3	도착 여유 < 생성 시간	due = 8s 앞 지점이 중심에서 420m 초과. FAR_HALF 680m. 180km/h(50m/s)면 남은 거리 약 260m = 약 5s. 단지 장면 생성은 이보다 길다. v>12m/s면 단지 대신 일반 영역만 선택하는 우회가 있다	월드 끝(빈 지면) 노출, 고속에서 상세 건물 사라짐
D4	예측이 직진 투영뿐	lead = max(260, 200+8v) 직진 지점 1개를 고정(heldPoint)한다. 도로 그래프와 배송 경로를 쓰지 않는다. 실패한 후보는 hopRefused에 쌓인다	교차로에서 꺾으면 선로딩 무효, 엉뚱한 방향 로딩
D5	두 장면 동시 상주	carry 동안 옛 traffic.tick이 계속 돌고 옛 모델도 그려진다(letGo 지연)	드로우·메모리 2배, 그 구간 fps 하락
D6	충돌 격자 재계산과 공백	startDrive마다(이관 포함) solidGrid(FAR_HALF=680, 0.5m) = 2720×2720 = 약 740만 셀을 메인 스레드에서 메시 삼각형 래스터로 채운다(frameSlice 분할). 완료 전 dv.solids = null	이관 직후 건물을 통과, 로딩 중 미세 끊김
D7	교통 재생성	새 영역마다 buildTraffic으로 차량 전체를 새로 생성하고 히어로를 summon한다	주변 차량 순간 교체(팝), 충돌 직후 상태 소실
D8	경로 탐색 비용	route()는 배열 선형 탐색 Dijkstra(O(V²))이며 메인 스레드에서 350ms(이탈 시)~1200ms마다 호출된다	도로가 많은 지역에서 주기적 끊김
D9	오디오 재생성	이관 때 dv.sound.dispose() 후 startDrive가 new EngineSound → AudioContext 재생성, 음원 fetch-decode를 다시 한다	엔진음 끊김, 반복 시 컨텍스트 누적 위험
D10	물리·렌더 결합	시뮬레이션이 렌더 프레임 안에서 dt(최대 100ms)로 진행된다. 프레임 드랍이 곧 조작감 저하로 이어진다	드랍 시 차량이 '미끄러지듯' 튼, 입력 지연
D11	주행 중 상세 반경 확대	setDetail(240m/100m): 모델 차량·번호판 반경 확대, near-mesh 매 프레임 재패킹	교통 많은 구간 CPU 증가
D12	구조적 복잡도	주행 로직이 4,781줄 React 컴포넌트에 결합. 상태 이관이 수작업	회귀 버그(재시작 연료 0 폭발 등) 반복

4. 재설계: 스트리밍 월드 아키텍처

그림 2. 제안 구조: 타일 단위 스트리밍 (주행은 어떤 단계도 기다리지 않음)



4.1 타일과 LOD 링

링	반경	내용	생성 위치
L0 근거리	≤ 350m	도로 면·차선·연석·가로등·가로수, 주변 건물(공유 외벽 아틀라스, 사전 생성 35종 재사용), 보행자, 충돌 데이터	워커 → 업로드 예산
L1 중거리	350~900m	건물 매스(동일 재질), 지형, 물, 도로 면	워커, 저비용
L2 원경	900m~2.5k m	HLOD 병합 실루엣 + 지평선(타일당 드로우 1회)	사전 베이킹(CDN)

- **타일 250m 고정 격자**(Web Mercator 또는 현재 쓰는 로컬 미터 좌표의 전역 격자). 키 = (tx, ty). 단지 중심 기준이 아니므로 재중심화가 없다.
- **부동 원점**: 차량이 원점에서 2km를 넘으면 모든 타일 그룹의 위치만 한 번에 이동한다(행렬 갱신 1회). 지오메트리를 다시 만들지 않는다.
- **링 전환은 고정 배분**: 카메라 거리에 따라 실시간으로 LOD를 바꾸면 팝·깜빡임이 생긴다('깜빡임 금지' 원칙). 타일 단위로 L1 → L0을 0.4초 디더 페이드로 올린다.
- **품질 원칙**: 근거리 L0은 현재 단지뷰의 근거리 품질(해상도·그림자·AO·TAA)을 그대로 쓴다. 원경 단순화는 화면상 크기에 맞춘 것으로, 해상도 하향이 아니다.

4.2 타일 데이터 계층

- 백엔드 **/api/drive/tile/{tx}/{ty}**: VWorld 건물(높이·윤곽), 도로(중심선·폭), 필지 용도, 물(OSM water 포함), DEM 격자를 한 번에 묶은 **바이너리 팩**(Float32/Int16 배열 + 인덱스, brotli)을 반환한다. 브라우저가 영역마다 VWorld JSONP를 여러 번 호출하던 구조를 대체한다.
- 서버 캐시(30일) + CDN 캐시 헤더 + 브라우저 IndexedDB(타일 LRU 400개). 한 번 지나간 길은 다음부터 네트워크 없이 열린다. VWorld 이용약관의 캐시·재배포 조건은 적용 전에 확인해야 한다.
- 자주 달리는 구역(서울 주요 단지 주변)은 배치 작업으로 미리 굽는다. 그 밖의 지역은 요청 시 생성한 뒤 캐시한다.

4.3 워커 생성 파이프라인

- **TileWorker 풀(2~3개)**: 디코드 → 지형 메시 → 도로 면/차선/연석(기존 sceneSidewalk·sceneStreet 알고리즘 이식) → 건물 압출·재질별 병합 → 가로수·가로등 인스턴스 행렬 → **2D 충돌 데이터** → 도로 그래프 조각을 만든다. 결과는 Transferable ArrayBuffer로 넘긴다.
- **충돌은 메시 래스터 대신 데이터 윤곽 사용**: 건물 footprint 폴리곤 + 타일별 균일 격자 해시(8m). 차량 OBB 대 폴리곤 SAT로 판정한다. 타일 생성과 함께 준비되므로 공백이 없고, 740만 셀 래스터가 사라진다(D6 해소).
- Sol이 만든 trafficCollision.ts(할당 없는 SAT)를 차량·건물 판정 공통 루틴으로 재사용한다.

4.4 GPU 업로드 스케줄러와 파이프라인

- 프레임당 **업로드 예산**(예: CPU 1.5ms 또는 4MB)과 **ETA 우선순위 큐**를 둔다. 큰 버퍼는 mappedAtCreation 청크로 나누고, 타일당 render bundle을 한 번 기록한다.
- **주행 전용 파이프라인 세트**: 타일 재질은 고정된 소수 조합(지면·도로·차선·건물 아틀라스·식재·차량)만 쓰도록 하고, 주행 시작 전에 createRenderPipelineAsync로 전부 컴파일해 둔다. 주행 중 새 셰이더 컴파일 0건을 단언으로 검사한다(GPU.syncCompiles = 0).
- rAF 간격이 22ms를 넘으면 예산을 절반으로 줄인다(frameSlice의 백오프 규칙과 동일). 예측 여유가 충분하므로 로딩이 늦어져도 화면은 끊기지 않는다.

4.5 예측 선로딩 (도로 그래프 기반 ETA)

선로딩 반경 $H = v \times (T_{\text{생성 } p95} + T_{\text{업로드}} + \text{여유 } 2s) + R_{L0}$. 50m/s, 생성 0.8s, 업로드 0.6s라면 $H = 50 \times 3.4 + 350 = \text{약 } 520\text{m}$. **후보 타일** = 현재 도로 간선에서 제한 Dijkstra(이진 힙)로 $ETA \leq T_{\text{horizon}}$ 안에 닿는 도로가 지나는 타일. **우선순위** = 배송 경로 위 타일(가중 0.3) → 진행 방향 $\pm 30^\circ$ → 그 밖의 도달 가능 타일 순, 동물이면 ETA가 짧은 쪽. 후보에서 빠진 타일 작업은 취소하고(AbortController + 워커 epoch), 이미 만든 타일은 LRU로 유지한다.

- 요청은 상위 8개만 동시에 진행한다. 교차로 직전에는 갈림길 양쪽을 모두 L1까지 미리 만들어 둔다(저비용)
- 매 프레임이 아니라 200ms마다, 그리고 간선이 바뀔 때 재계산한다. 경로 탐색은 워커에서 하고 메인 스레드는 결과만 받는다(D4-D8 해소).
- 안전망**: L0이 도착 전에 준비되지 못하면 L1(매스)로라도 반드시 덮는다. L1은 비용이 작아 반경을 크게 잡을 수 있다. 빈 지면은 절대 보이지 않는다.

4.6 영속 시스템: 교통 · 충돌 · 물리 · 오디오

시스템	현재	재설계
교통	영역마다 buildTraffic 재생성, 히어로 summon 이관	전역 1개 교통. 도로 그래프에 타일 간선을 추가/제거하는 증분 방식. 차량은 L0 경계 밖(시야 밖)에서 생성하고 뒤쪽에서 소멸. 교차 감시(waitOn 그래프)는 유지
경로	$O(V^2)$ Dijkstra, 메인 스레드, 350~1200ms	이진 힙 A*(거리 휴리스틱), 워커. 경로 변경 시에만 리본 갱신
충돌	SolidGrid 재래스터(공백 발생)	타일별 footprint 해시. 타일과 함께 생성되고 해제되며 공백 없음
물리	렌더 프레임 dt(최대 100ms)	고정 120Hz 누적기 + 렌더 보간. 고속에서는 이동 경로 스왑 판정(CCD)으로 터널링 방지. 장기적으로 물리·교통을 SimWorker로 분리
오디오	이관마다 AudioContext 재생성	세션당 AudioContext 1개, 디코드 버퍼 캐시, 차량 교체 시 크로스페이드
상태	carry/startDrive 재실행 수작업 이관	DriveSession 객체(게임·연료·HP·골드)를 월드와 분리. 월드가 바뀌어도 세션은 그대로

4.7 렌더링 최적화 (주행 모드)

- 타일 단위 프러스텀·거리 컬링과 타일당 render bundle로 드로우 제출 비용을 타일 수에 비례하게 만든다.
- 그림자 캐스케이드: 원경 캐스케이드는 N프레임마다 갱신하고 근경만 매 프레임 갱신한다(현 그림자 캐시 확장). 반사 큐브는 주행 중 일정 간격으로 더블버퍼 교체한다.
- TAA 재투영이 고속 카메라에서 잔상을 만들지 점검하고, 필요하면 모션 벡터 기반 재투영과 이력 거부 기준을 강화한다.
- 교통 near-mesh: 변경 슬롯만 갱신. 상세 반경 240m는 화면상 크기 기준(픽셀 높이 $\geq N$)으로 바꾼다. 품질은 같고 불필요한 모델 차량이 줄어든다.
- 장기: 렌더러 워커(OffscreenCanvas). 메인 스레드의 React/HUD 갱신이 주행 화면 프레임에 영향을 주지 않게 한다.

4.8 단지 상세 업그레이드 총

현재 단지뷰의 강점(실측 3D 형상, 사진 분석 창호, 단지 색)은 그대로 살린다. 차량이 단지 150m 안에서 저속($\leq 8\text{m/s}$)이거나 배송 목적지일 때, 그 단지만 기존 파이프라인으로 상세 모델을 만들어 해당 타일 건물과 0.5초 페이드로 교체한다. 고속 통과 때는 만들지 않는다. 그래서 '고속 시 단지 상세 사라짐'은 결함이 아니라 의도된 설계가 되고, 배송지 도착 순간에는 항상 최고 품질이 보인다.

5. 게임 기능 추가 제안

우선	기능	내용	서비스 연계 / 비고
P0	조작·안정성	게임패드(Gamepad API), 터치 조향 개선(기울기 옵션), 키 재매핑, 일시정지 메뉴, 체크포인트 재시작	모바일 접근성
P0	임장 드라이브	배송 도착 시 단지 카드(실거레가·세대수·주차, K-apt) 팝업, '이 단지 3D로 보기' 연결	부동산 지도와 직접 연계, 체류시간 증대
P1	미션 모드	연속 배송 콤보(시간 내 연속 성공 시 배수), 타임어택 체크포인트, 택시 승객 픽업·하차, 야간 배송(조명·시야)	기존 시간대·날씨 슬라이더 활용
P1	교통 규칙	신호 위반·과속·역주행 감점, 단속 카메라, 보행자 보호구역(학교 앞) 속도 제한	기존 신호 시스템 재사용
P1	성장 요소	골드로 차량 업그레이드(가속·제동·연비·내구), 차량 도색, 신규 차량 해금(버스·택시·오토바이)	차량 모델 20여 종 보유
P2	소셜	일일 시드 챌린지(전원 같은 출발·목적지), 지역별 리더보드, 고스트 리플레이(입력 기록 → 재생)	Turso drive_scores 확장
P2	공유	포토 모드·하이라이트 클립(MediaRecorder), 링크 공유 시 같은 경로 도전	기존 3D 공유 링크 구조
P2	몰입	날씨별 노면 그림(비 = 제동거리 증가), 피해 시각화 확장, 라디오(지역 뉴스·시황 TTS)	주식 시황 연계 가능

6. 단계별 실행 계획

단계	기간(추정)	작업	완료 기준
0. 즉시 개선	1주	D6 충돌을 footprint 해시로 교체(공백 제거) · AudioContext 1개 유지 · route()를 이진 힙 A*로 · due 판정을 생성 시간 실측 기반 여유로 · 주행 진입 영역은 외벽 페인트·식재·보행자 생략(L1 수준) · carry 중 옛 장면 tick 중단	이관 후 건물 통과 0, 음 끊김 0, 180km/h 월드 끝 노출 감소
1. 타일 MVP	2~3주	타일 API·바이너리 팩·IndexedDB, TileWorker(지형·도로·건물 매스·충돌), 부동 원점, L1 링, 업로드 스케줄러, 사전 컴파일 세트	5km 자동 주행 p99 ≤ 25ms, 월드 끝 0
2. 완전 스트리밍	2~3주	L0 상세(차선·가로수·가로등·보행자), 영속 교통 증분 그래프, ETA 예측기, 고정 스텝 물리, 단지 상세 업그레이드 층	p99 ≤ 20ms, 4배 감속 ≤ 33ms, 30분 메모리 평탄
3. 게임 확장	2주~	P0/P1 기능, 공개 전환(devgame 해제) 판단	플레이 테스트, 점수 집계 정상
4. 장기	3주~	렌더러·시뮬레이션 워커 분리, L2 HLOD 사전 베이킹	입력 지연 p99 ≤ 16ms

7. 검증 계획: 드라이브 벤치마크

- **자동 주행 하네스**: DEV 혹(_holoStage.drive/traffic)으로 고정 경로(길음·개포·마곡 각 5km, 교차로 회전 포함)를 30/60/120/180km/h로 주행한다. 키 입력 대신 목표 경로 추종 컨트롤러를 쓴다.
- **지표**: rAF 간격 p50/p95/p99/최대, 33ms 초과 횟수, '월드 끝 근접' 이벤트(차량 → 미준비 타일 거리 < 100m), 타일 ETA 여유 분포, 주행 중 동기 컴파일 수, 업로드 바이트/프레임, 메모리(10분 간격), 오디오 언더런.
- **조건**: 프로덕션 빌드(vite preview), 기준선과 같은 시점 A/B ×3, CDP CPU 4배 감속, 네트워크 Fast 3G 프로파일, WebGL 강제 경로.
- **판정 스크립트**: check-drive-targets.py는 기준 미달 시 exit 1을 반환한다. 기능 통과와 성능 통과를 분리 보고한다(Sol이 확립한 방식 유지).
- 교통 교착 회귀는 기존 방식(traffic.update 600회 × 30분 시뮬)을 타일 경계를 넘나드는 경로로 확장해 검사한다.

8. 리스크

- **데이터 약관:** VWorld 데이터를 서버·CDN에 캐시하거나 재배포할 수 있는지 확인이 필요하다. 불가하면 서버 캐시 없이 브라우저 IndexedDB만 쓰고 서버는 중계만 한다.
- **시각 일관성:** 타일 경계에서 도로·지형 이음새가 생길 수 있다. 도로는 타일 경계를 넘는 원본 선형으로 생성하고 소유 타일만 그리며, 지형은 경계 정점을 공유한다.
- **범위:** 단지뷰(정지 감상)와 주행(스트리밍)의 두 모드가 같은 렌더러를 공유하므로, 주행 전용 경로는 기능 플래그(?stream=1)로 병행 개발한 뒤 전환한다.
- **작업 트리 정리:** 연속 주행 WIP가 미커밋 상태로 다른 영역 변경과 섞여 있다. 0단계 시작 전에 별도 브랜치로 분리하기를 권한다.